

GESTÃO E QUALIDADE DE SOFTWARE

SPRINT 3 — RELATÓRIO FINAL

Este é o documento final do projeto. Ele acompanha a apresentação e a demo ao vivo. Registre aqui não só o que foi construído, mas o que o grupo aprendeu ao longo do semestre.

1. Identificação

Nome do Projeto	Ecommerce Checkout Engine
URL do Repositório	https://github.com/MPicolli/ecommerce-checkout-engine
URL da Demo (se houver)	
Integrantes	Eric Levi Sena Silveira (RA 10723114930) José Augusto Ferreira (RA 10725220369) Leonardo de Medeiros Binatti (RA 1072322655) Matheus Picolli Ishibashi (RA 10724262102) Vithor Massing dos Santos (RA 1072316868)
Data da Apresentação	17/06/2026

2. Checklist de Entrega

Marque o que o grupo está entregando. O professor vai verificar cada item durante a apresentação.

Item	Status
Repositório no GitHub ou Codeberg com código completo e histórico de commits	☒ Sim ☐ Não
README com instruções claras de como instalar e rodar o projeto	☒ Sim ☐ Não
Testes automatizados passando (rode antes da apresentação)	☒ Sim ☐ Não
Relatório Final (este documento) preenchido	☒ Sim ☐ Não
Demo preparada (ao vivo ou gravada com aval prévio do professor)	☒ Sim ☐ Não

3. O que Foi Construído

3.1 Funcionalidades entregues

Liste todas as funcionalidades que fazem parte da versão final do projeto.

#	Funcionalidade	Tem teste?	Funciona na demo?
---	----------------	------------	-------------------

1	Modelagem do domínio (Produto, Cliente, Carrinho, ItemCarrinho, Cupom, ResumoPedido e TipoCupom) com validações defensivas	☒ Sim ☐ Não	☒ Sim ☐ Não
2	Serviço de cálculo de frete por CEP e peso dos produtos (FreteService)	☒ Sim ☐ Não	☒ Sim ☐ Não
3	Serviço de validação e aplicação de cupons de desconto (CupomService)	☒ Sim ☐ Não	☒ Sim ☐ Não
4	Serviço de acúmulo e resgate de pontos de fidelidade (FidelidadeService)	☒ Sim ☐ Não	☒ Sim ☐ Não
5	Processamento completo do checkout e geração do ResumoPedido (CheckoutService)	☒ Sim ☐ Não	☒ Sim ☐ Não
6	Interface gráfica em JavaFX para demonstração do fluxo completo de compra	☐ Sim ☒ Não	☒ Sim ☐ Não

3.2 O que ficou de fora do escopo original e por quê

Alguma funcionalidade planejada não foi implementada?

Durante o desenvolvimento do projeto, o grupo considerou a implementação de funcionalidades complementares, como persistência de dados por meio de DAOs, operações de CRUD para gerenciamento das entidades do sistema e verificação de estoque disponível por produto. No entanto, essas funcionalidades não faziam parte do escopo mínimo definido para a disciplina e acabaram não sendo implementadas.

A principal razão foi a necessidade de priorizar a conclusão das funcionalidades centrais do checkout, a cobertura de testes automatizados e a documentação técnica do projeto. Além disso, dificuldades de planejamento e comunicação em alguns momentos da Sprint reduziram o tempo disponível para desenvolver funcionalidades adicionais com a qualidade desejada.

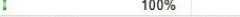
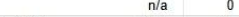
Em contrapartida, foi implementada uma interface gráfica utilizando JavaFX (v23.0.2), FXML e CSS Inline para facilitar a demonstração do fluxo completo do sistema durante a apresentação final. A adoção do JavaFX representou uma mudança em relação à abordagem inicial de interface em console.

4. Qualidade e Testes

4.1 Cobertura final

Total de testes	Cobertura (%)	Unitários	Integração

checkout-engine

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.unisul.ecommerce.repository		0%		0%	12 12	32 32	11 11	3 3
com.unisul.ecommerce		0%		n/a	3 3	9 9	3 3	1 1
com.unisul.ecommerce.service		100%		100%	0 53	0 122	0 18	0 4
com.unisul.ecommerce.model		100%		100%	0 89	0 160	0 67	0 7
com.unisul.ecommerce.exception		100%		n/a	0 3	0 6	0 3	0 3
Total	248 of 1.300	80%	2 of 116	98%	15 160	41 329	14 102	4 18

4.2 Decisões de qualidade

Quais decisões técnicas o grupo tomou para garantir a qualidade do código?

Para garantir a qualidade do código, o projeto foi estruturado com separação clara de responsabilidades entre entidades de domínio, serviços e exceções customizadas. As regras de negócio foram centralizadas em serviços independentes, permitindo a criação de testes unitários isolados utilizando JUnit 5 e Mockito.

Também foram adotadas práticas como programação defensiva nas entidades, uso de BigDecimal para operações monetárias, validação de entradas, tratamento explícito de erros por meio de exceções específicas e documentação técnica das regras de negócio por meio de especificações BDD, cenários de teste e documentação arquitetural.

No controle de versão, foram utilizadas branches por funcionalidade, Pull Requests e commits padronizados. Embora essas práticas tenham sido aplicadas durante boa parte do projeto, o grupo identificou que nem sempre o processo de revisão de código foi seguido de forma consistente, o que representa uma oportunidade de melhoria para trabalhos futuros.

5. Controle de Versão

Total de commits no projeto

Aproximadamente 70 commits

Branches utilizadas

docs/documentacao-tecnica; feature/testes-modelos; feature/testes-checkoutservice; feature/testes-cupomservice; feature/testes-freteservice

CI/CD utilizado?

☐ Sim (qual ferramenta: _____) ☒ Não

Todos os integrantes commitaram?

☒ Sim ☐ Não (explique: _____)

As branches das Sprints anteriores estão registradas nos respectivos relatórios. As branches listadas acima referem-se exclusivamente ao trabalho realizado na Sprint 3.

6. Reflexão do Grupo

6.1 O que o grupo aprendeu na prática

Quais conceitos da disciplina ficaram mais claros depois de aplicar no projeto?

O projeto permitiu compreender na prática que qualidade de software vai muito além de escrever código que funciona. Ao longo do semestre ficou evidente que aspectos como organização do trabalho, comunicação entre os integrantes, documentação, testes automatizados e revisão de código possuem impacto direto na qualidade do resultado final.

O grupo também percebeu a importância de definir regras de negócio de forma clara antes da implementação. Em diversos momentos foi necessário revisar decisões tomadas anteriormente para remover ambiguidades e alinhar o comportamento do sistema.

Outro aprendizado importante foi que ferramentas como testes automatizados, cobertura de código e controle de versão ajudam a reduzir erros, mas dependem da disciplina da equipe para gerar resultados consistentes. Quando os processos não são seguidos por todos os integrantes, torna-se mais difícil manter a qualidade e a rastreabilidade do desenvolvimento.

6.2 O que faria diferente

Se o projeto recomeçasse hoje, o que o grupo mudaria nas decisões técnicas ou no processo?

Se o projeto fosse reiniciado hoje, o grupo investiria mais esforço no planejamento das atividades e na comunicação entre os integrantes desde o início das sprints. Uma das principais dificuldades encontradas foi concentrar parte significativa do trabalho próximo às datas de entrega, o que reduziu o tempo disponível para revisões mais cuidadosas e para a implementação de funcionalidades adicionais planejadas. O grupo também reforçaria o cumprimento do fluxo de desenvolvimento definido inicialmente, especialmente em relação às revisões obrigatórias de Pull Requests antes do merge. Embora a prática tenha sido adotada em diversos momentos, ela não foi seguida com a consistência esperada ao longo de todo o projeto. Além disso, a decisão de adotar JavaFX para a interface gráfica, ao invés da interface no terminal, demandou esforço de adaptação que poderia ter sido melhor absorvido se a escolha tivesse sido definida desde o início da Sprint. Em projetos futuros, o grupo buscaria fechar as decisões de stack antes de iniciar a implementação.

6.3 O projeto tem futuro?

O software que vocês construíram tem potencial de continuar? O que seria necessário para levá-lo adiante?

O projeto cumpriu principalmente um papel educacional, servindo como laboratório para aplicação prática dos conceitos estudados na disciplina. Embora a arquitetura construída seja coesa e permita evolução, o sistema atual ainda representa uma versão simplificada de um motor de checkout.

Para que pudesse evoluir para um cenário mais próximo de produção, seria necessário implementar persistência de dados, operações de CRUD, verificação de estoque, integração com banco de dados, autenticação de usuários e testes de integração mais abrangentes. A interface gráfica em JavaFX construída para a demo poderia servir de ponto de partida para uma aplicação desktop funcional, caso o grupo optasse por continuar o desenvolvimento.

Mesmo sem perspectiva de continuidade, o projeto demonstrou o valor concreto das práticas de qualidade de software. A experiência de trabalhar com testes automatizados, documentação técnica versionada e controle de versão colaborativo fornece uma base que todos os integrantes podem levar para projetos reais no futuro.

Bom trabalho ao longo do semestre.